Le Cycle de Vie des Composants et la Gestion des Effets Secondaires

Objectifs

1. Comprendre les différentes phases du cycle de vie d'un composant fonctionnel (montage, mise à jour, démontage).
2. Maîtriser l'utilisation du Hook `useEffect` pour synchroniser un composant avec un système extérieur.
3. Configurer correctement le tableau de dépendances pour éviter les boucles infinies et les fuites de mémoire.
4. Implémenter une fonction de nettoyage (cleanup) pour les abonnements et les minuteurs.

1. Comprendre le Cycle de Vie dans React

Contrairement aux composants à base de classes qui disposaient de méthodes spécifiques (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`), les composants fonctionnels vivent au rythme des **rendus** successifs.

Le cycle de vie d'un composant fonctionnel se décompose en trois grandes phases :

1. **Le Montage (Mount) :** Le composant est créé et affiché pour la toute première fois dans le DOM réel.
2. **Le Rendu et la Mise à Jour (Update) :** Le composant se ré-exécute suite à un changement de ses props ou de son état local (`useState`), mettant à jour l'interface si nécessaire.
3. **Le Démontage (Unmount) :** Le composant est retiré de la page (fermeture d'un panneau, changement de route).

Qu'est-ce qu'un Effet Secondaire (Side Effect) ?

Dans une application React, le code situé directement dans le corps de la fonction du composant doit être une **fonction pure** : il calcule le JSX à retourner en fonction des props et de l'état, sans modifier l'extérieur.

Toute opération ayant un impact sur le monde extérieur ou nécessitant une interaction avec le système (appels API réseau, manipulation directe du DOM, abonnements à des WebSockets, utilisation de `setInterval` ou `localStorage`) constitue un **effet secondaire**. Ces opérations doivent être isolées à l'intérieur du Hook `useEffect`.

---

2. Anatomie et Syntaxe du Hook `useEffect`

Le Hook `useEffect` prend deux arguments :

1. **Une fonction de rappel (effect callback)** contenant le code à exécuter.
2. **Un tableau de dépendances optionnel** qui contrôle le moment où l'effet doit se déclencher.

```
None
import { useState, useEffect } from 'react';

export default function StationMonitor() {
  const [telemetry, setTelemetry] = useState(null);

  useEffect(() => {
    // Code exécuté après le rendu du composant
    console.log('Le composant vient d être monté ou mis à jour.');
  }); // Attention : sans tableau de dépendances, s'exécute à CHAQUE rendu !

  return <div>Moniteur de station</div>;
}
```

---

3. Le Rôle Crucial du Tableau de Dépendances

Le comportement de `useEffect` change radicalement selon la façon dont vous renseignez son second argument (le tableau de dépendances) :

3.1 Absence de Tableau de Dépendances

```
None
useEffect(() => {
  // Exécuté après le premier rendu ET après CHAQUE re-rendu du composant
});
```

Cas d'usage : Rarement recommandé, utilisé uniquement pour des opérations de traçage de débogage.

3.2 Tableau Vide (`[ ]`)

```
None
useEffect(() => {
  // Exécuté EXACTEMENT UNE SEULE FOIS, au moment du montage initial du composant
  fetchStationData();
}, []);
```

Cas d'usage : Chargement initial de données depuis une API, initialisation d'un écouteur global unique, récupération de paramètres au démarrage.

3.3 Tableau avec Dépendances (`[id, status]`)

```
None
useEffect(() => {
  // Exécuté au montage, PUIS à chaque fois que la variable 'stationId'
  // ou 'status' change de valeur entre deux rendus.
  loadStationDetails(stationId);
}, [stationId, status]);
```

Cas d'usage : Recharger des données dès qu'un identifiant ou un filtre change dans l'interface.

---

4. La Fonction de Nettoyage (Cleanup)

Certains effets secondaires mettent en place des ressources persistantes (abonnements réseau, minuteurs, écouteurs d'événements globaux). Si le composant est démonté sans que ces ressources soient libérées, cela provoque des fuites de mémoire (*memory leaks*) ou des erreurs d'exécution (*Cannot update a unmounted component*).

Pour éviter cela, la fonction passée à `useEffect` peut retourner une **fonction de nettoyage**. React l'exécutera juste avant que le composant ne soit démonté, ainsi qu'avant d'exécuter l'effet lors du rendu suivant si les dépendances ont changé.

None

```
import { useState, useEffect } from 'react';

export default function LiveClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    // 1. Mise en place de la ressource (Minuteur)
    const timerId = setInterval(() => {
      setTime(new Date());
      console.log('Tic-tac...');
    }, 1000);

    // 2. Fonction de nettoyage retournée
    return () => {
      clearInterval(timerId);
      console.log('Minuteur nettoyé et arrêté.');
    };
  }, []); // Exécuté une seule fois au montage

  return (
    <div className="card">
      <p>Heure système : {time.toLocaleTimeString()}</p>
    </div>
  );
}
```

---

5. Pièges Fréquents et Bonnes Pratiques

1. **Oublier des dépendances dans le tableau :** Si votre effet utilise une variable ou une fonction définie dans le composant, elle doit obligatoirement figurer dans le tableau de dépendances, sous peine d'utiliser des valeurs obsolètes (*stale closures*).

2. **Déclencher une boucle infinie :** Modifier un état à l'intérieur d'un `useEffect` sans tableau de dépendances (ou dont la dépendance est précisément l'état modifié) déclenchera un re-rendu, qui relancera l'effet, créant une boucle infinie fatale.

3. **Utiliser `async/await` directement sur l'effet :** La fonction de rappel de `useEffect` ne peut pas être déclarée directement comme `async`, car React attend qu'elle retourne soit `undefined`, soit une fonction de nettoyage. Il faut définir une fonction asynchrone interne et l'appeler immédiatement.

None

```
// BONNE PRATIQUE : Appel d'une fonction asynchrone interne
useEffect(() => {
  async function loadData() {
    try {
      const response = await fetch('/api/stations');
      const data = await response.json();
      setStations(data);
    } catch (error) {
      console.error('Erreur réseau :', error);
    }
  }

  loadData();
}, []);
```

---

6. Synthèse

1. **Cycle de Vie :** Un composant fonctionnel traverse les phases de montage, de mise à jour et de démontage au rythme des rendus de React.
2. **Effets Secondaires :** Le Hook `useEffect` isole les interactions avec l'extérieur du corps pur du composant.
3. **Maîtrise des Dépendances :** Tableau vide pour un montage unique, tableau rempli pour réagir à des changements ciblés, absence de tableau pour un effet à chaque rendu.
4. **Nettoyage :** Retourner une fonction de nettoyage permet de libérer les ressources et d'éviter les fuites mémoire lors du démontage.